

## Código fuente — PC4 Desarrollo de Software

Repositorio: <https://github.com/Melissa1221/pc4-desarrollo-software>

Estructura del proyecto:

```
pom.xml
src/main/java/com/uni/petfinder/alertas/AlertCenter.java
src/main/java/com/uni/petfinder/alertas/AlertObserver.java
src/main/java/com/uni/petfinder/alertas/LostPet.java
src/main/java/com/uni/petfinder/alertas/Sighting.java
src/main/java/com/uni/petfinder/App.java
src/main/java/com/uni/petfinder/busqueda/AdoptionSearch.java
src/main/java/com/uni/petfinder/busqueda/ImageQuery.java
src/main/java/com/uni/petfinder/busqueda/LostCheckSearch.java
src/main/java/com/uni/petfinder/busqueda/SaleSearch.java
src/main/java/com/uni/petfinder/busqueda/SearchIntent.java
src/main/java/com/uni/petfinder/busqueda/SearchStrategy.java
src/main/java/com/uni/petfinder/busqueda/SearchStrategyFactory.java
src/main/java/com/uni/petfinder/cuidadores/BaseCaregiver.java
src/main/java/com/uni/petfinder/cuidadores/CaregiverProfile.java
src/main/java/com/uni/petfinder/cuidadores/CaregiverProfileBuilder.java
src/main/java/com/uni/petfinder/cuidadores/CaregiverRole.java
src/main/java/com/uni/petfinder/cuidadores/ProfessionalRole.java
src/main/java/com/uni/petfinder/cuidadores/RoleDecorator.java
src/main/java/com/uni/petfinder/cuidadores/SolidarityRole.java
src/main/java/com/uni/petfinder/cuidadores/SpecializedRole.java
```

### ./pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.uni</groupId>
  <artifactId>petfinder</artifactId>
  <version>1.0</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <build>
    <plugins>
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.1.0</version>
        <configuration>
          <mainClass>com.uni.petfinder.App</mainClass>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

```

        </plugins>
    </build>
</project>

```

## src/main/java/com/uni/petfinder/alertas/AlertCenter.java

```

package com.uni.petfinder.alertas;

import java.util.ArrayList;
import java.util.List;

// Singleton: registro central de alertas. Es el Subject del patrón Observer.
public class AlertCenter {

    private static AlertCenter instance;

    // observador -> ubicación donde escucha (para el radio del RF 1.4).
    private final List<Subscription> subs = new ArrayList<>();

    private AlertCenter() {
    }

    public static AlertCenter getInstance() {
        if (instance == null) {
            instance = new AlertCenter();
        }
        return instance;
    }

    public void subscribe(AlertObserver obs, double lat, double lon) {
        subs.add(new Subscription(obs, lat, lon));
    }

    public void unsubscribe(AlertObserver obs) {
        subs.removeIf(s -> s.obs == obs);
    }

    // RF 1.4: al reportar una mascota se notifica solo a los que están
    // dentro del radio configurable (km).
    public void reportLostPet(LostPet pet, double radiusKm) {
        for (Subscription s : subs) {
            if (distanceKm(pet.getLat(), pet.getLon(), s.lat, s.lon) <= radiusKm) {
                s.obs.onLostPet(pet);
            }
        }
    }

    // distancia aproximada en km entre dos coordenadas.
    private double distanceKm(double lat1, double lon1, double lat2, double lon2) {
        double dLat = (lat1 - lat2) * 111;
        double dLon = (lon1 - lon2) * 111;
        return Math.sqrt(dLat * dLat + dLon * dLon);
    }

    private static class Subscription {

```

```

        final AlertObserver obs;
        final double lat;
        final double lon;

        Subscription(AlertObserver obs, double lat, double lon) {
            this.obs = obs;
            this.lat = lat;
            this.lon = lon;
        }
    }
}

```

#### src/main/java/com/uni/petfinder/alertas/AlertObserver.java

```

package com.uni.petfinder.alertas;

// Observer: quien quiera recibir alertas de mascotas perdidas.
public interface AlertObserver {
    void onLostPet(LostPet pet);
}

```

#### src/main/java/com/uni/petfinder/alertas/LostPet.java

```

package com.uni.petfinder.alertas;

// RF 1.1: mascota reportada como perdida por su dueño.
public class LostPet {

    private final String name;
    private final String species;
    private final String breed;
    private final String photo;
    private final String description;
    private final String ownerId;
    private final double lat;
    private final double lon;

    public LostPet(String name, String species, String breed, String photo,
                   String description, String ownerId, double lat, double lon) {
        this.name = name;
        this.species = species;
        this.breed = breed;
        this.photo = photo;
        this.description = description;
        this.ownerId = ownerId;
        this.lat = lat;
        this.lon = lon;
    }

    public String getName() {
        return name;
    }

    public String getSpecies() {

```

```

        return species;
    }

    // RF 1.2: coordenadas del reporte.
    public double getLat() {
        return lat;
    }

    public double getLon() {
        return lon;
    }

    // RNF 1.2: el id del dueño no se expone al ciudadano que avista.
    public String getOwnerId() {
        return ownerId;
    }

    public String getDescription() {
        return description;
    }
}

```

#### src/main/java/com/uni/petfinder/alertas/Sighting.java

```

package com.uni.petfinder.alertas;

// RF 1.3: avistamiento registrado por un ciudadano anónimo.
public class Sighting {

    private final String photo;
    private final double lat;
    private final double lon;

    public Sighting(String photo, double lat, double lon) {
        this.photo = photo;
        this.lat = lat;
        this.lon = lon;
    }

    public String getPhoto() {
        return photo;
    }

    public double getLat() {
        return lat;
    }

    public double getLon() {
        return lon;
    }
}

```

## src/main/java/com/uni/petfinder/App.java

```
package com.uni.petfinder;

import com.uni.petfinder.alertas.AlertCenter;
import com.uni.petfinder.alertas.AlertObserver;
import com.uni.petfinder.alertas.LostPet;
import com.uni.petfinder.busqueda.ImageQuery;
import com.uni.petfinder.busqueda.SearchIntent;
import com.uni.petfinder.busqueda.SearchStrategy;
import com.uni.petfinder.busqueda.SearchStrategyFactory;
import com.uni.petfinder.cuidadores.BaseCaregiver;
import com.uni.petfinder.cuidadores.CaregiverProfile;
import com.uni.petfinder.cuidadores.CaregiverProfileBuilder;
import com.uni.petfinder.cuidadores.CaregiverRole;
import com.uni.petfinder.cuidadores.ProfessionalRole;
import com.uni.petfinder.cuidadores.SpecializedRole;

import java.util.List;

// Demo que ejercita los tres módulos con sus patrones.
public class App {

    public static void main(String[] args) {

        System.out.println("=== Módulo 1: alertas (Observer + Singleton) ===");
        AlertCenter center = AlertCenter.getInstance();

        // un cuidador se suscribe a las alertas de su zona.
        AlertObserver ana = pet ->
            System.out.println("Ana recibe alerta: " + pet.getName() + " perdido cerca");
        center.subscribe(ana, -12.021, -77.031);

        // otro cuidador está lejos, no debería recibir la alerta.
        AlertObserver luis = pet ->
            System.out.println("Luis recibe alerta: " + pet.getName());
        center.subscribe(luis, -12.20, -76.90);

        LostPet firulais = new LostPet("Firulais", "perro", "criollo", "foto.jpg",
            "collar rojo", "owner-1", -12.02, -77.03);
        center.reportLostPet(firulais, 1.0); // radio 1 km

        System.out.println("\n=== Módulo 2: buscador por imagen (Strategy + Factory) ===");
        ImageQuery query = new ImageQuery("mascota.png");
        query.put("species", "perro");
        for (SearchIntent intent : SearchIntent.values()) {
            SearchStrategy strategy = SearchStrategyFactory.create(intent);
            System.out.println("Intención " + intent + ":");
            for (String r : strategy.search(query)) {
                System.out.println("  - " + r);
            }
        }

        System.out.println("\n=== Módulo 3: cuidadores (Builder + Decorator) ===");
        CaregiverProfile profile = new CaregiverProfileBuilder()
            .name("María")
```

```

        .acceptSpecies("perro")
        .acceptSpecies("gato")
        .acceptSize("pequeño")
        .givesMedication(true)
        .verifyId(true)
        .build();
System.out.println("Perfil de " + profile.getName()
    + " | público: " + profile.isPublic()
    + " | especies: " + profile.getAcceptedSpecies());

// roles combinados con Decorator.
CaregiverRole role = new SpecializedRole(new ProfessionalRole(new BaseCaregiver("María")))
System.out.println(role.description());

// RF 3.3: interruptor de alertas.
profile.toggleAlerts(false);
System.out.println("¿Recibe alertas? " + profile.receiveAlerts());
    }
}

```

#### src/main/java/com/uni/petfinder/busqueda/AdoptionSearch.java

```

package com.uni.petfinder.busqueda;

import java.util.Arrays;
import java.util.List;

// RF 2.3: en adopción solo se devuelven resultados de protectoras/ONGs.
public class AdoptionSearch implements SearchStrategy {

    @Override
    public List<String> search(ImageQuery query) {
        return Arrays.asList(
            "ONG Patitas Felices - perro similar en adopción",
            "Protectora San Roque - perro similar en adopción");
    }
}

```

#### src/main/java/com/uni/petfinder/busqueda/ImageQuery.java

```

package com.uni.petfinder.busqueda;

// RF 2.1 / RNF 2.1: consulta que llega al buscador. Los metadatos van en
// un mapa (JSON plano) para que el motor sea intercambiable.
import java.util.HashMap;
import java.util.Map;

public class ImageQuery {

    private final String imageFile;
    private final Map<String, String> metadata = new HashMap<>();

    public ImageQuery(String imageFile) {
        this.imageFile = imageFile;
    }
}

```

```

    }

    public String getImageFile() {
        return imageFile;
    }

    public void put(String key, String value) {
        metadata.put(key, value);
    }

    public String get(String key) {
        return metadata.get(key);
    }
}

```

### src/main/java/com/uni/petfinder/busqueda/LostCheckSearch.java

```

package com.uni.petfinder.busqueda;

import java.util.ArrayList;
import java.util.List;

// RF 2.5: verificar pérdida contrasta los metadatos con las alertas activas.
public class LostCheckSearch implements SearchStrategy {

    @Override
    public List<String> search(ImageQuery query) {
        List<String> matches = new ArrayList<>();
        String species = query.get("species");
        if (species != null) {
            matches.add("Alerta activa que coincide en especie: " + species);
        } else {
            matches.add("Sin coincidencias en alertas activas");
        }
        return matches;
    }
}

```

### src/main/java/com/uni/petfinder/busqueda/SaleSearch.java

```

package com.uni.petfinder.busqueda;

import java.util.Arrays;
import java.util.List;

// RF 2.4: en venta solo se muestran criaderos comerciales certificados.
public class SaleSearch implements SearchStrategy {

    @Override
    public List<String> search(ImageQuery query) {
        return Arrays.asList(
            "Criadero Los Andes (certificado) – ejemplar en venta",
            "Criadero Norte (certificado) – ejemplar en venta");
    }
}

```

```
}
```

### src/main/java/com/uni/petfinder/busqueda/SearchIntent.java

```
package com.uni.petfinder.busqueda;

// RF 2.2: el usuario está obligado a elegir una de estas tres intenciones.
public enum SearchIntent {
    ADOPCION,
    VENTA,
    VERIFICAR_PERDIDA
}
```

### src/main/java/com/uni/petfinder/busqueda/SearchStrategy.java

```
package com.uni.petfinder.busqueda;

import java.util.List;

// Strategy: cada intención de búsqueda resuelve la consulta a su manera.
public interface SearchStrategy {
    List<String> search(ImageQuery query);
}
```

### src/main/java/com/uni/petfinder/busqueda/SearchStrategyFactory.java

```
package com.uni.petfinder.busqueda;

// Factory: entrega la estrategia de búsqueda según la intención elegida.
public class SearchStrategyFactory {

    public static SearchStrategy create(SearchIntent intent) {
        switch (intent) {
            case ADOPCION:
                return new AdoptionSearch();
            case VENTA:
                return new SaleSearch();
            case VERIFICAR_PERDIDA:
                return new LostCheckSearch();
            default:
                throw new IllegalArgumentException("Intención no soportada");
        }
    }
}
```

### src/main/java/com/uni/petfinder/cuidadores/BaseCaregiver.java

```
package com.uni.petfinder.cuidadores;

// Componente concreto base sobre el que se añaden los roles.
public class BaseCaregiver implements CaregiverRole {
```

```

    private final String name;

    public BaseCaregiver(String name) {
        this.name = name;
    }

    @Override
    public String description() {
        return name + ": cuidador";
    }
}

```

### src/main/java/com/uni/petfinder/cuidadores/CaregiverProfile.java

```

package com.uni.petfinder.cuidadores;

import java.util.ArrayList;
import java.util.List;

// Producto del Builder: perfil de un cuidador con sus restricciones.
public class CaregiverProfile {

    private String name;
    private List<String> acceptedSpecies = new ArrayList<>();
    private List<String> acceptedSizes = new ArrayList<>();
    private boolean givesMedication;
    private boolean idVerified; // RNF 3.1
    private boolean receivesAlerts; // RF 3.3

    public String getName() {
        return name;
    }

    public List<String> getAcceptedSpecies() {
        return acceptedSpecies;
    }

    public List<String> getAcceptedSizes() {
        return acceptedSizes;
    }

    public boolean givesMedication() {
        return givesMedication;
    }

    // RNF 3.1: el perfil solo se habilita si el documento fue validado.
    public boolean isPublic() {
        return idVerified;
    }

    // RF 3.3: interruptor de alertas del módulo 1.
    public boolean receivesAlerts() {
        return receivesAlerts;
    }
}

```

```

public void toggleAlerts(boolean on) {
    this.receivesAlerts = on;
}

// acceso de paquete: solo el builder arma el perfil.
void setName(String name) {
    this.name = name;
}

void addSpecies(String s) {
    acceptedSpecies.add(s);
}

void addSize(String s) {
    acceptedSizes.add(s);
}

void setMedication(boolean b) {
    this.givesMedication = b;
}

void setIdVerified(boolean b) {
    this.idVerified = b;
}
}

```

#### src/main/java/com/uni/petfinder/cuidadores/CaregiverProfileBuilder.java

```

package com.uni.petfinder.cuidadores;

// Builder: arma el perfil paso a paso. RF 3.2 permite definir restricciones
// de servicio que son opcionales, por eso conviene un builder.
public class CaregiverProfileBuilder {

    private final CaregiverProfile profile = new CaregiverProfile();

    public CaregiverProfileBuilder name(String name) {
        profile.setName(name);
        return this;
    }

    public CaregiverProfileBuilder acceptSpecies(String species) {
        profile.addSpecies(species);
        return this;
    }

    public CaregiverProfileBuilder acceptSize(String size) {
        profile.addSize(size);
        return this;
    }

    public CaregiverProfileBuilder givesMedication(boolean value) {
        profile.setMedication(value);
        return this;
    }
}

```

```

    // RNF 3.1: marca el documento como validado.
    public CaregiverProfileBuilder verifyId(boolean value) {
        profile.setIdVerified(value);
        return this;
    }

    public CaregiverProfile build() {
        return profile;
    }
}

```

#### src/main/java/com/uni/petfinder/cuidadores/CaregiverRole.java

```

package com.uni.petfinder.cuidadores;

// Component del Decorator: un rol de cuidador describe qué servicios ofrece.
public interface CaregiverRole {
    String description();
}

```

#### src/main/java/com/uni/petfinder/cuidadores/ProfessionalRole.java

```

package com.uni.petfinder.cuidadores;

public class ProfessionalRole extends RoleDecorator {

    public ProfessionalRole(CaregiverRole wrapped) {
        super(wrapped);
    }

    @Override
    public String description() {
        return wrapped.description() + " + profesional (cobra por servicio)";
    }
}

```

#### src/main/java/com/uni/petfinder/cuidadores/RoleDecorator.java

```

package com.uni.petfinder.cuidadores;

// Decorator: cada rol envuelve a otro y suma capacidades. RF 3.1 define los
// tres roles (solidario, profesional, especializado).
public abstract class RoleDecorator implements CaregiverRole {

    protected CaregiverRole wrapped;

    public RoleDecorator(CaregiverRole wrapped) {
        this.wrapped = wrapped;
    }

    @Override
    public String description() {
        return wrapped.description();
    }
}

```

```
}  
}
```

#### src/main/java/com/uni/petfinder/cuidadores/SolidarityRole.java

```
package com.uni.petfinder.cuidadores;  
  
public class SolidarityRole extends RoleDecorator {  
  
    public SolidarityRole(CaregiverRole wrapped) {  
        super(wrapped);  
    }  
  
    @Override  
    public String description() {  
        return wrapped.description() + " + solidario (cuidado básico gratuito)";  
    }  
}
```

#### src/main/java/com/uni/petfinder/cuidadores/SpecializedRole.java

```
package com.uni.petfinder.cuidadores;  
  
public class SpecializedRole extends RoleDecorator {  
  
    public SpecializedRole(CaregiverRole wrapped) {  
        super(wrapped);  
    }  
  
    @Override  
    public String description() {  
        return wrapped.description() + " + especializado (medicación y casos delicados)";  
    }  
}
```